



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Análisis Numérico para la Ingeniería - CE1111

Informe 4 Problema 1

Profesor: Juan Pablo Soto Quirós

Alumnos:

Mauro Andrés Navarro Obando

Paula Melina Porras Mora

Grupo: 1

27 abril 2026

0.1. Estrategia de implementación

El método de Jacobi construye una sucesión de matrices similares a A mediante rotaciones ortogonales sucesivas, con el objetivo de convertirla en una matriz diagonal cuyos elementos corresponden a los valores propios. En cada iteración se identifica el elemento fuera de la diagonal de mayor valor absoluto, ubicado en la posición (i_m, j_m) , y se calcula el ángulo de rotación θ que lo anularía:

$$\theta = \frac{1}{2} \arctan \left(\frac{2A_{ij}}{A_{ii} - A_{jj}} \right) \quad (1)$$

Con este ángulo se construye la matriz de rotación de Jacobi $G = I_n + Z$, donde Z es nula salvo en las posiciones:

$$Z_{ii} = Z_{jj} = \cos(\theta) - 1 \quad (2)$$

$$Z_{ij} = -\sin(\theta), \quad Z_{ji} = \sin(\theta) \quad (3)$$

Luego, se actualiza la matriz mediante:

$$A_{k+1} = G^\top A_k G \quad (4)$$

Dado que G es ortogonal, esta transformación preserva los valores propios. Repitiendo el proceso, los elementos fuera de la diagonal se reducen progresivamente hasta que la diagonal converge a los valores propios de A .

0.2. Criterio de convergencia

Se utilizó el criterio basado en la variación del vector de valores propios entre iteraciones consecutivas, definido como:

$$e_k = \|x_{k+1} - x_k\|_1 \quad (5)$$

donde $x_k = \text{diag}(A_k)$ es el vector de elementos diagonales en la iteración k . El método se detiene cuando:

$$e_k < 10^{-10} \quad (6)$$

o cuando se alcanza el número máximo de iteraciones permitido.

0.3. Comandos y funciones utilizados

La búsqueda del elemento máximo fuera de la diagonal se realizó construyendo la matriz:

$$|A_k - \text{diag}(\text{diag}(A_k))| \quad (7)$$

y aplicando las funciones `np.argmax` junto con `np.unravel_index` para obtener los índices (i_m, j_m) . El ángulo se calculó con `np.arctan`, y las funciones trigonométricas `np.cos` y `np.sin` se usaron para construir la matriz de rotación mediante `np.eye(n)`.

La actualización $G^\top A_k G$ se realizó con el operador `@`. El error se calculó mediante:

$$\text{np.linalg.norm}(\mathbf{xk_new} - \mathbf{xk}, \text{ord}=1) \quad (8)$$

y la diagonal se extrajo con la función `np.diag(Ak)`.

0.4. Convergencia y número de iteraciones

El método convergió con un error final de:

$$4,43 \times 10^{-12} \quad (9)$$

muy por debajo de la tolerancia establecida de 10^{-10} . Los resultados muestran dos valores propios no nulos:

$$\lambda_1 \approx 128,19 \quad (10)$$

$$\lambda_{15} \approx -8,19 \quad (11)$$

mientras que los trece restantes son aproximadamente cero. Esto es correcto, dado que la matriz definida por:

$$A_{ij} = 0,5(i + j) \quad (12)$$

tiene rango 2.

La convergencia fue estable, favorecida por la simetría de la matriz, propiedad que el método de Jacobi explota directamente al garantizar que cada rotación reduce la norma de Frobenius de la parte fuera de la diagonal.

0.5. Resultados

En la Fig. 1 se presentan los resultados obtenidos mediante la implementación computacional del método de Jacobi.

```
Valores propios aproximados:
[ 1.28190908e+02  1.71742294e-12  1.14299702e-12  6.23863340e-13
 2.54212978e-13  1.52780130e-13  1.30897554e-13  1.21252532e-13
 8.52972668e-14  4.87041862e-14  4.06493654e-16  3.05066087e-16
-6.56760733e-17 -3.10891348e-11 -8.19090848e+00]

Error final: 4.4305442269985324e-12
```

Figura 1: Resultados obtenidos mediante el método de Jacobi.